Maximum SubArray Sum (Kadane's Algorithm) (10)

contigous part of the array

arr = [-2, -3, 4, -1, -2, 1, 5, -3]

o/p

7

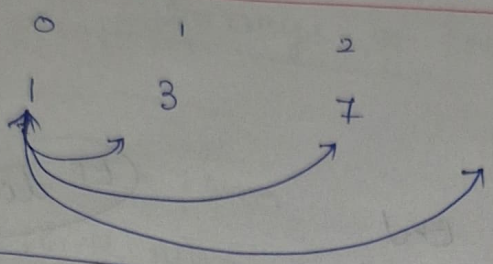
✓ -3 → is a sub.A

✓ 4, -1 → " " "

✓ But 4, -1, 1 ↓ It is a subseq

App 1 → to try all subArray (BruteForce Approach)

For eg:




```
for(int i=0 ; i<size ; i++)
{
 for(int j=i ; j<size ; j++)
  {
  for(int k=i ; k<=j ; k++)
   {
    Sys(a[k]
   }
  }
}
```

endpt = 0,0 ⇒ 1

intially starting will be the endpoint

i=0 → [1] , 0<4 ✓

j=0 → [1] , 0<4 ✓

i → to iterate to the next next num, (starting i=0 till → length)

[1] next [3] next [7] next [8]

j → to iterate next next num of each individual num above, (to determine the range)

i → starting from 1 till 8

j = i till → length

k → to print the num

k = i (starting) till the num in current j

dry run

i = 0 ; 0 < 4 (length) ✓

j = 0 ; 0 < 4 ✓ (j = i)

k = 0 ; k <= 0 ✓ (k = i, k <= j) {1}

sys(a[k])

a[0] => 1

then k++ → k = 1 ① k <= 0 ✗ (gets out of k loop)

then j++ → j = 1

j = 1 1 < 4 ✓ (j < leng)

k = 0 k <= 1 (k = i, k <= j) (to print from 0 till the current j)

syso a[k] => 0

k++, k = 1 k <= 1 ✓

a[k] => 1

{1, 3}

Then ktoop gets fail and goes to j loop
Current j is updated to 2, so then goes to k loop then print from 0 till current j
{0, 1, 2} - {1, 3, 7}

Maximum SubArray Sum

```
for(int i=0; i<size; i++)
{
for(int j=i ; j<size; j++)
  {
   for (int k=i; k<=j ; k++)
   {
    sum = sum + a[k]
   }
   if (sum > max)
   {
    max = sum
   }
   sum=0
  }
}
```

nearly $O(N^3)$

App 2: Better approach

→ Every time in K loops, calculates the sum starting num till the current j (which is already calculated sum for prev subarray) except current j.

→ In this Better approach, we found that The sum of the current subarray is, i.e [Sum of previous subarray] + new encountered num

1 → 1

1 3 → [previous.s](1) + 3 = 4

1 3 7 → [previous.s](4) + 7 = 11

1 3 7 8 → [previous.s](11) + 8 = 19

```
for (int i=0 ; i<size; i++)
{       sum=0
 for(int j=i; j<size ; j++)
    {
      sum = sum + a[j]
      max = Math.max(sum, max)
    }
}
syso (max)
```

O(N2)

dry run

sum = 0 (initially)

i = 0 → 0 < 4 ✓ (i = 0, i < size)

j = 0 → j < 4 ✓ (j = i, j < size) [1]

sum = 0 + a[j]{1} ⇒ sum = 0 + 1 = [1]

then j++ → j = 1

---

j = 1 → 1 < 4 ✓ [1, 3]

↳ newly encountered index of new subarray

sum = (1) + a[j]{3} ⇒ sum = 1 + 3 = [4]

↳ sum of previous subarray

then j++ → j = 2

---

j = 2 → 2 < 4

↳ newly encountered index of new subarray

sum = (4) + a[j]{7} = 4 + 7 = [11]

↳ sum of previous subarray

then j++ → j = 3

j = 3 → 3 < 4 ✓

sum = 11 + a[j]{8} = 11 + 8 = 19

j++ → j = 4 ✗

then i = 1 → sum = 0 → bcoz going to next num 3

App 3: Optimized approach (Kadane's Algorithm)

arr[] = {-2, -3, 4, -1, -2, 1, 5, -3}

→ keep on moving forward, when sum < 0. If not (when adding the new encountered num to pre. sum) drop it out & Move to next idx. and make sum = 0

because there is no point in forward with negative num. It decrease the summation value

Intially
max = Integer.minValue

sum	max
sum = -2 check if (sum<0) ✗ so i++ (jump to next) & sum=0	Max = Math.max(-2, max) (Integer.minValue) max = -2
sum = -3 check if (sum<0) ✗ so i++ & make sum = 0	Max = Math.max(-3, -2) max = -2
sum = 4 check if (4<0) ← i++ sum = 4	Math.max(4, -2) max = 4
~~(if (sum>0) = sum = sum+a[i])~~ sum = 4 - 1 = 3 i++ check if (3<0) ← sum = 3 → keep moving with this num. still it is not -ve num	(3, 4) max = 4

sum	max
sum = 3 - 2 = 1	max(-1, 4)
if(-1<0) ~~x~~ ✓	max = 4
sum = 0	
i++	
sum = 1 + 1	max(2, 4)
if(2<0) ✓	max = 4
sum = 2	
i++	
sum = 2 + 5	max(7, 4)
sum = 7	max = 7
i++	
Condition meet	

```
for(int i=0; i<size; i++)
{
  sum = sum + a[i]
  max = Math.max(sum, max)
  if(sum<0)
  {
    sum = 0
  }
}
```

O(N)

To print the subArray that has Maximum SubArray

Whenever There is sum == 0 (It is a start)

starting a new subArray

→ add new variable

ansStart = -1

ansEnd = -1

Whenever sum = 0 (I am starting the subArray)

```
for(int i=0; i<size; i++)
{
    Sum = sum + arr[i]
    if (sum > max)
    {
        max = sum;
        ansStart = start;
        ansEnd = i;
    }
    if(sum<0)
    {
        sum=0
    }
}
```

sum = 0 → if i am starting a new subArray then start = i

is sum the value this after starting new subArray

current i,

i.e. i am starting the new subArray

dry run {-2, -3, 4, -1, -2, 1, 5, -3}

```
for(int i=0; i<size; i++)
{
  sum = sum + arr[i]

  if (sum > max)
  {
    max = sum
    andstart = start
    andEnd = i
  }
  if (sum<0)
  { sum =0
    start = i
  }
}
```

intially sum = 0 (so starting)

sum = -2

if (-2 > Integer.MIN_VALUE)

{

max = -2

andstart = 0 (starting of subArray)

andEnd = i(0) (till now the subArray started is continuing)

}